In [1]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
```

In [2]:
```
data = pd.read_excel("../data/online_retail_II.xlsx")
```

In [3]:
```
df = data
```

In [4]:
```
df.head()
```

Out[4]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
0	489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	2009-12-01 07:45:00	6.95	13085.0	United Kingdom
1	489434	79323P	PINK CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.0	United Kingdom
2	489434	79323W	WHITE CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.0	United Kingdom
3	489434	22041	RECORD FRAME 7" SINGLE SIZE	48	2009-12-01 07:45:00	2.10	13085.0	United Kingdom
4	489434	21232	STRAWBERRY CERAMIC TRINKET BOX	24	2009-12-01 07:45:00	1.25	13085.0	United Kingdom

EXPLORATORY DATA ANALYSIS

In [5]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 525461 entries, 0 to 525460
Data columns (total 8 columns):
 #   Column       Non-Null Count   Dtype
---  ------       --------------   -----
 0   Invoice      525461 non-null  object
 1   StockCode    525461 non-null  object
 2   Description  522533 non-null  object
 3   Quantity     525461 non-null  int64
 4   InvoiceDate  525461 non-null  datetime64[us]
 5   Price        525461 non-null  float64
 6   Customer ID  417534 non-null  float64
 7   Country      525461 non-null  str
dtypes: datetime64[us](1), float64(2), int64(1), object(3), str(1)
memory usage: 32.1+ MB
```

In [6]:
```
df.isna().sum()
```

Out[6]:
```
Invoice             0
StockCode           0
Description      2928
Quantity            0
InvoiceDate         0
Price               0
Customer ID    107927
Country             0
dtype: int64
```

There are some missing values present in the dataset.

In [7]:
```
df.describe()
```

Out[7]:

	Quantity	InvoiceDate	Price	Customer ID
count	525461.000000	525461	525461.000000	417534.000000
mean	10.337667	2010-06-28 11:37:36.845018	4.688834	15360.645478
min	-9600.000000	2009-12-01 07:45:00	-53594.360000	12346.000000
25%	1.000000	2010-03-21 12:20:00	1.250000	13983.000000
50%	3.000000	2010-07-06 09:51:00	2.100000	15311.000000
75%	10.000000	2010-10-15 12:45:00	4.210000	16799.000000
max	19152.000000	2010-12-09 20:01:00	25111.090000	18287.000000
std	107.424110	NaN	146.126914	1680.811316

There are some negative values in Quantity and Price column which is not ideal. We will investigate it further.

In [8]:
```
df.describe(include="O")
```

```
C:\Users\mzama\AppData\Local\Temp\ipykernel_39076\504318277.py:1: Pandas4Warning:
For backward compatibility, 'str' dtypes are included by select_dtypes when 'obje
ct' dtype is specified. This behavior is deprecated and will be removed in a futu
re version. Explicitly pass 'str' to `include` to select them, or to `exclude` to
remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-mig
ration-select-dtypes for details on how to write code that works with pandas 2 an
d 3.
  df.describe(include="O")
```

Out[8]:

	Invoice	StockCode	Description	Country
count	525461	525461	522533	525461
unique	28816	4632	4681	40
top	537434	85123A	WHITE HANGING HEART T-LIGHT HOLDER	United Kingdom
freq	675	3516	3549	485852

In [9]:
```
df[df["Customer ID"].isna()].head(10)
```

Out[9]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Count
263	489464	21733	85123a mixed	-96	2009-12-01 10:52:00	0.00	NaN	Unite Kingdo
283	489463	71477	short	-240	2009-12-01 10:52:00	0.00	NaN	Unite Kingdo
284	489467	85123A	21733 mixed	-192	2009-12-01 10:53:00	0.00	NaN	Unite Kingdo
470	489521	21646	NaN	-50	2009-12-01 11:44:00	0.00	NaN	Unite Kingdo
577	489525	85226C	BLUE PULL BACK RACING CAR	1	2009-12-01 11:49:00	0.55	NaN	Unite Kingdo
578	489525	85227	SET/6 3D KIT CARDS FOR KIDS	1	2009-12-01 11:49:00	0.85	NaN	Unite Kingdo
1055	489548	22271	FELTCRAFT DOLL ROSIE	1	2009-12-01 12:32:00	2.95	NaN	Unite Kingdo
1056	489548	22254	FELT TOADSTOOL LARGE	12	2009-12-01 12:32:00	1.25	NaN	Unite Kingdo
1057	489548	22273	FELTCRAFT DOLL MOLLY	3	2009-12-01 12:32:00	2.95	NaN	Unite Kingdo
1058	489548	22195	LARGE HEART MEASURING SPOONS	1	2009-12-01 12:32:00	1.65	NaN	Unite Kingdo

When we dont have a customer, there is no point trying to cluster that record. So we will eventually drop these columns where customer ID is missing.

In [10]:
```
df[df["Quantity"] < 0].head(10)
```

Out[10]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Count
178	C489449	22087	PAPER BUNTING WHITE LACE	-12	2009-12-01 10:33:00	2.95	16321.0	Austra
179	C489449	85206A	CREAM FELT EASTER EGG BASKET	-6	2009-12-01 10:33:00	1.65	16321.0	Austra
180	C489449	21895	POTTING SHED SOW 'N' GROW SET	-4	2009-12-01 10:33:00	4.25	16321.0	Austra
181	C489449	21896	POTTING SHED TWINE	-6	2009-12-01 10:33:00	2.10	16321.0	Austra
182	C489449	22083	PAPER CHAIN KIT RETRO SPOT	-12	2009-12-01 10:33:00	2.95	16321.0	Austra
183	C489449	21871	SAVE THE PLANET MUG	-12	2009-12-01 10:33:00	1.25	16321.0	Austra
184	C489449	84946	ANTIQUE SILVER TEA GLASS ETCHED	-12	2009-12-01 10:33:00	1.25	16321.0	Austra
185	C489449	84970S	HANGING HEART ZINC T-LIGHT HOLDER	-24	2009-12-01 10:33:00	0.85	16321.0	Austra
186	C489449	22090	PAPER BUNTING RETRO SPOTS	-12	2009-12-01 10:33:00	2.95	16321.0	Austra
196	C489459	90200A	PURPLE SWEETHEART BRACELET	-3	2009-12-01 10:44:00	4.25	17592.0	Unit Kingdc

The dataset described that if the invoice start with "C", it means its a cancelled order

In [11]:
```
df['Invoice'].nunique()
```

Out[11]: 28816

Lets identify if all the entries in the invoice column obey the laws of dataset description where the number must be 6 digits long and there can either be a "C" or nothing on the front.

In [12]:
```
df["Invoice"] = df["Invoice"].astype("str")
```

In [13]:
```
df[df["Invoice"].str.match("^C?\d{6}$") == True]
```

```
<>:1: SyntaxWarning: invalid escape sequence '\d'
<>:1: SyntaxWarning: invalid escape sequence '\d'
C:\Users\mzama\AppData\Local\Temp\ipykernel_39076\199291292.py:1: SyntaxWarning:
invalid escape sequence '\d'
  df[df["Invoice"].str.match("^C?\d{6}$") == True]
```

Out[13]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Co
0	489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	2009-12-01 07:45:00	6.95	13085.0	U King
1	489434	79323P	PINK CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.0	U King
2	489434	79323W	WHITE CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.0	U King
3	489434	22041	RECORD FRAME 7" SINGLE SIZE	48	2009-12-01 07:45:00	2.10	13085.0	U King
4	489434	21232	STRAWBERRY CERAMIC TRINKET BOX	24	2009-12-01 07:45:00	1.25	13085.0	U King
...	...	...	...	...	...	...	...	
525456	538171	22271	FELTCRAFT DOLL ROSIE	2	2010-12-09 20:01:00	2.95	17530.0	U King
525457	538171	22750	FELTCRAFT PRINCESS LOLA DOLL	1	2010-12-09 20:01:00	3.75	17530.0	U King
525458	538171	22751	FELTCRAFT PRINCESS OLIVIA DOLL	1	2010-12-09 20:01:00	3.75	17530.0	U King
525459	538171	20970	PINK FLORAL FELTCRAFT SHOULDER BAG	2	2010-12-09 20:01:00	3.75	17530.0	U King
525460	538171	21931	JUMBO STORAGE BAG SUKI	2	2010-12-09 20:01:00	1.95	17530.0	U King

525458 rows × 8 columns

In [14]:
```
df[df["Invoice"].str.match("^C?\d{6}$") == False]
```

```
<>:1: SyntaxWarning: invalid escape sequence '\d'
<>:1: SyntaxWarning: invalid escape sequence '\d'
C:\Users\mzama\AppData\Local\Temp\ipykernel_39076\679444801.py:1: SyntaxWarning:
invalid escape sequence '\d'
  df[df["Invoice"].str.match("^C?\d{6}$") == False]
```

Out[14]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID
179403	A506401	B	Adjust bad debt	1	2010-04-29 13:36:00	-53594.36	NaN
276274	A516228	B	Adjust bad debt	1	2010-07-19 11:24:00	-44031.79	NaN
403472	A528059	B	Adjust bad debt	1	2010-10-20 12:04:00	-38925.87	NaN

These are some sort of balancing the accounts debts.

In [15]:
```
incorrect_stockcode = df[df["StockCode"].str.match("^\d{5}$") == False]
```

```
<>:1: SyntaxWarning: invalid escape sequence '\d'
<>:1: SyntaxWarning: invalid escape sequence '\d'
C:\Users\mzama\AppData\Local\Temp\ipykernel_39076\4064361616.py:1: SyntaxWarning:
invalid escape sequence '\d'
  incorrect_stockcode = df[df["StockCode"].str.match("^\d{5}$") == False]
```

In [16]:
```
incorrect_stockcode.shape
```

Out[16]:
```
(80112, 8)
```

These appear to be ligit transactions as the price and customer ids are available. As nothing is mentioned in the dataset about this, we are going to consider this ligit and proceed with them.

In [17]:
```
df[df["StockCode"] == "PADS"]
```

Out[17]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Co
62299	494914	PADS	PADS TO MATCH ALL CUSHIONS	1	2010-01-19 17:04:00	0.001	16705.0	Kir
74731	496222	PADS	PADS TO MATCH ALL CUSHIONS	1	2010-01-29 13:53:00	0.001	13583.0	Kir
77702	496473	PADS	PADS TO MATCH ALL CUSHIONS	1	2010-02-01 15:38:00	0.001	17350.0	Kir
79794	496643	PADS	PADS TO MATCH ALL CUSHIONS	1	2010-02-03 11:58:00	0.001	13408.0	Kir
90798	497935	PADS	PADS TO MATCH ALL CUSHIONS	1	2010-02-15 10:47:00	0.001	13408.0	Kir
97716	498562	PADS	PADS TO MATCH ALL CUSHIONS	1	2010-02-21 12:03:00	0.001	15182.0	Kir
101718	499056	PADS	PADS TO MATCH ALL CUSHIONS	1	2010-02-24 13:46:00	0.001	13765.0	Kir
104480	499399	PADS	PADS TO MATCH ALL CUSHIONS	1	2010-02-26 13:26:00	0.001	14459.0	Kir
123947	501176	PADS	PADS TO MATCH ALL CUSHIONS	1	2010-03-15 11:00:00	0.001	14857.0	Kir
156809	504332	PADS	PADS TO MATCH ALL CUSHIONS	1	2010-04-12 16:30:00	0.001	12671.0	Ge
178629	506303	PADS	PADS TO MATCH ALL CUSHIONS	1	2010-04-29 10:43:00	0.001	14249.0	Kir
249035	C513477	PADS	PADS TO MATCH ALL CUSHIONS	-1	2010-06-24 16:56:00	36.600	14459.0	Kir
341826	522766	PADS	PADS TO MATCH ALL CUSHIONS	1	2010-09-16 13:58:00	0.001	15182.0	Kir
440930	531433	PADS	PADS TO MATCH ALL CUSHIONS	1	2010-11-08 11:54:00	0.001	13317.0	Kir
468280	533528	PADS	PADS TO MATCH ALL CUSHIONS	1	2010-11-17 15:08:00	0.001	15750.0	Kir

What if we just drop the customer ID missing columns and check that how many of these values are eliminated.

In [18]:
```
missing_customer_id = df[df['Customer ID'].isna()]
```

In [19]:
```
missing_customer_id.shape
```

Out[19]:
```
(107927, 8)
```

In [20]:
```
incorrect_stockcode["StockCode"].isin(missing_customer_id['StockCode']).sum()
```

Out[20]:
```
np.int64(79137)
```

It appears that majority of the entries that have an incorrect stockcode is majorly a business entry and not a customer entry as they are missing the customer ids in the dataset.

DATA CLEANING

In [21]:
```
cleaned_df = df.copy()
```

In [22]:
```
# Clean the Invoice column

cleaned_df["Invoice"] = cleaned_df["Invoice"].astype("str")

mask = (
    cleaned_df["Invoice"].str.match("^\\d{6}$") == True
)

cleaned_df = cleaned_df[mask]

cleaned_df
```

Out[22]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Co
0	489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	2009-12-01 07:45:00	6.95	13085.0	U King
1	489434	79323P	PINK CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.0	U King
2	489434	79323W	WHITE CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.0	U King
3	489434	22041	RECORD FRAME 7" SINGLE SIZE	48	2009-12-01 07:45:00	2.10	13085.0	U King
4	489434	21232	STRAWBERRY CERAMIC TRINKET BOX	24	2009-12-01 07:45:00	1.25	13085.0	U King
...	...	...	...	...	...	...	...	
525456	538171	22271	FELTCRAFT DOLL ROSIE	2	2010-12-09 20:01:00	2.95	17530.0	U King
525457	538171	22750	FELTCRAFT PRINCESS LOLA DOLL	1	2010-12-09 20:01:00	3.75	17530.0	U King
525458	538171	22751	FELTCRAFT PRINCESS OLIVIA DOLL	1	2010-12-09 20:01:00	3.75	17530.0	U King
525459	538171	20970	PINK FLORAL FELTCRAFT SHOULDER BAG	2	2010-12-09 20:01:00	3.75	17530.0	U King
525460	538171	21931	JUMBO STORAGE BAG SUKI	2	2010-12-09 20:01:00	1.95	17530.0	U King

515252 rows × 8 columns

In [23]:

```
# Clean The StockCode column

cleaned_df["StockCode"] = cleaned_df["StockCode"].astype("str")

mask = (
    (cleaned_df["StockCode"].str.match("^\\d{5}$") == True)
    | (cleaned_df["StockCode"].str.match("^\\d{5}[a-zA-Z]+$") == True)
    | (cleaned_df["StockCode"].str.match("^PADS$") == True)
)

cleaned_df = cleaned_df[mask]
```

```
cleaned_df
```

Out[23]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Co
0	489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	2009-12-01 07:45:00	6.95	13085.0	U Kin
1	489434	79323P	PINK CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.0	U Kin
2	489434	79323W	WHITE CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.0	U Kin
3	489434	22041	RECORD FRAME 7" SINGLE SIZE	48	2009-12-01 07:45:00	2.10	13085.0	U Kin
4	489434	21232	STRAWBERRY CERAMIC TRINKET BOX	24	2009-12-01 07:45:00	1.25	13085.0	U Kin
...	...	...	...	...	...	...	...	
525456	538171	22271	FELTCRAFT DOLL ROSIE	2	2010-12-09 20:01:00	2.95	17530.0	U Kin
525457	538171	22750	FELTCRAFT PRINCESS LOLA DOLL	1	2010-12-09 20:01:00	3.75	17530.0	U Kin
525458	538171	22751	FELTCRAFT PRINCESS OLIVIA DOLL	1	2010-12-09 20:01:00	3.75	17530.0	U Kin
525459	538171	20970	PINK FLORAL FELTCRAFT SHOULDER BAG	2	2010-12-09 20:01:00	3.75	17530.0	U Kin
525460	538171	21931	JUMBO STORAGE BAG SUKI	2	2010-12-09 20:01:00	1.95	17530.0	U Kin

512796 rows × 8 columns

In [24]:
```
cleaned_df.describe()
```

Out[24]:

	Quantity	InvoiceDate	Price	Customer ID
count	512796.000000	512796	512796.000000	406337.000000
mean	11.001279	2010-06-28 18:26:52.709732	3.391447	15373.632992
min	-9600.000000	2009-12-01 07:45:00	0.000000	12346.000000
25%	1.000000	2010-03-21 13:27:00	1.250000	14004.000000
50%	3.000000	2010-07-06 14:25:00	2.100000	15326.000000
75%	10.000000	2010-10-15 14:50:00	4.210000	16814.000000
max	19152.000000	2010-12-09 20:01:00	1157.150000	18287.000000
std	104.352433	NaN	5.067864	1677.365907

In [25]:
```
# Clean the customer id column

cleaned_df.dropna(subset="Customer ID", inplace=True)
```

In [26]:
```
cleaned_df.describe()
```

Out[26]:

	Quantity	InvoiceDate	Price	Customer ID
count	406337.000000	406337	406337.000000	406337.000000
mean	13.619099	2010-07-01 10:11:06.543287	2.991462	15373.632992
min	1.000000	2009-12-01 07:45:00	0.000000	12346.000000
25%	2.000000	2010-03-26 14:01:00	1.250000	14004.000000
50%	5.000000	2010-07-09 15:48:00	1.950000	15326.000000
75%	12.000000	2010-10-14 17:09:00	3.750000	16814.000000
max	19152.000000	2010-12-09 20:01:00	295.000000	18287.000000
std	97.000659	NaN	4.285876	1677.365907

Quantity is also solved. Lets look at price problem.

In [27]:
```
len(cleaned_df[cleaned_df["Price"] == 0])
```

Out[27]: 28

0 Priced item must be not be included in a sensible analysis, and as these transactions are only 28 we can drop them.

In [28]:
```
cleaned_df = cleaned_df[cleaned_df['Price'] > 0]
```

In [29]:
```
cleaned_df.describe()
```

Out[29]:

	Quantity	InvoiceDate	Price	Customer ID
count	406309.000000	406309	406309.000000	406309.000000
mean	13.617924	2010-07-01 10:14:25.869572	2.991668	15373.722268
min	1.000000	2009-12-01 07:45:00	0.001000	12346.000000
25%	2.000000	2010-03-26 14:01:00	1.250000	14006.000000
50%	5.000000	2010-07-09 15:48:00	1.950000	15326.000000
75%	12.000000	2010-10-14 17:09:00	3.750000	16814.000000
max	19152.000000	2010-12-09 20:01:00	295.000000	18287.000000
std	96.998833	NaN	4.285951	1677.329470

How much data did we loose during our cleaning proceed ?

In [30]:
```
current_data = round(len(cleaned_df) * 100 / len(df))
lost_data = round(100 - current_data)

print("Current Data: ", current_data)
print("Lost Data: ", lost_data )
```

```
Current Data:  77
Lost Data:  23
```

Dropped Almost 23% of entire records.

FEATURE ENGINEERING

We will create some columns for Measureing RFM for all the customers.

In [31]:
```
cleaned_df['TotalAmount'] = cleaned_df["Quantity"] * cleaned_df['Price']
cleaned_df.head()
```

Out[31]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
0	489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	2009-12-01 07:45:00	6.95	13085.0	United Kingdom
1	489434	79323P	PINK CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.0	United Kingdom
2	489434	79323W	WHITE CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.0	United Kingdom
3	489434	22041	RECORD FRAME 7" SINGLE SIZE	48	2009-12-01 07:45:00	2.10	13085.0	United Kingdom
4	489434	21232	STRAWBERRY CERAMIC TRINKET BOX	24	2009-12-01 07:45:00	1.25	13085.0	United Kingdom

Now we will bring our data on customer level by aggregating so that we can calculate RFM for different customers.

In [32]:
```
aggregated_df = cleaned_df.groupby("Customer ID", as_index=False) \
        .agg(
            MonetaryValue = ("TotalAmount", "sum"),
            Frequency=("Invoice", "nunique"),
            LastInvoiceDate =("InvoiceDate", "max")
        )

aggregated_df
```

Out[32]:

	Customer ID	MonetaryValue	Frequency	LastInvoiceDate
0	12346.0	169.36	2	2010-06-28 13:53:00
1	12347.0	1323.32	2	2010-12-07 14:57:00
2	12348.0	221.16	1	2010-09-27 14:59:00
3	12349.0	2221.14	2	2010-10-28 08:23:00
4	12351.0	300.93	1	2010-11-29 15:23:00
...	...	...	...	...
4280	18283.0	641.77	6	2010-11-22 15:30:00
4281	18284.0	411.68	1	2010-10-04 11:33:00
4282	18285.0	377.00	1	2010-02-17 10:24:00
4283	18286.0	1246.43	2	2010-08-20 11:57:00
4284	18287.0	2295.71	4	2010-11-22 11:51:00

4285 rows × 4 columns

In [33]:
```
max_invoice_date = aggregated_df["LastInvoiceDate"].max()

max_invoice_date
```

Out[33]:
```
Timestamp('2010-12-09 20:01:00')
```

In real implementation we usually take the todays data as that makes more sense.

In [34]:
```
aggregated_df["RecencyValue"] = (max_invoice_date - aggregated_df['LastInvoiceDa
```

In [35]:
```
aggregated_df
```

Out[35]:

	Customer ID	MonetaryValue	Frequency	LastInvoiceDate	RecencyValue
0	12346.0	169.36	2	2010-06-28 13:53:00	164
1	12347.0	1323.32	2	2010-12-07 14:57:00	2
2	12348.0	221.16	1	2010-09-27 14:59:00	73
3	12349.0	2221.14	2	2010-10-28 08:23:00	42
4	12351.0	300.93	1	2010-11-29 15:23:00	10
...	...	...	...	...	...
4280	18283.0	641.77	6	2010-11-22 15:30:00	17
4281	18284.0	411.68	1	2010-10-04 11:33:00	66
4282	18285.0	377.00	1	2010-02-17 10:24:00	295
4283	18286.0	1246.43	2	2010-08-20 11:57:00	111
4284	18287.0	2295.71	4	2010-11-22 11:51:00	17

4285 rows × 5 columns

Visualization

In [36]:
```
help(sns.histplot)
```

```
Help on function histplot in module seaborn.distributions:

histplot(
    data=None,
    *,
    x=None,
    y=None,
    hue=None,
    weights=None,
    stat='count',
    bins='auto',
    binwidth=None,
    binrange=None,
    discrete=None,
    cumulative=False,
    common_bins=True,
    common_norm=True,
    multiple='layer',
    element='bars',
    fill=True,
    shrink=1,
    kde=False,
    kde_kws=None,
    line_kws=None,
    thresh=0,
    pthresh=None,
    pmax=None,
    cbar=False,
    cbar_ax=None,
    cbar_kws=None,
    palette=None,
    hue_order=None,
    hue_norm=None,
    color=None,
    log_scale=None,
    legend=True,
    ax=None,
    **kwargs
)
    Plot univariate or bivariate histograms to show distributions of datasets.

    A histogram is a classic visualization tool that represents the distribution
    of one or more variables by counting the number of observations that fall wit
hin
    discrete bins.

    This function can normalize the statistic computed within each bin to estimat
e
    frequency, density or probability mass, and it can add a smooth curve obtaine
d
    using a kernel density estimate, similar to :func:`kdeplot`.

    More information is provided in the :ref:`user guide <tutorial_hist>`.

    Parameters
    ----------
    data : :class:`pandas.DataFrame`, :class:`numpy.ndarray`, mapping, or sequenc
e
        Input data structure. Either a long-form collection of vectors that can b
e
```

```
        assigned to named variables or a wide-form dataset that will be internall
y
        reshaped.
    x, y : vectors or keys in ``data``
        Variables that specify positions on the x and y axes.
    hue : vector or key in ``data``
        Semantic variable that is mapped to determine the color of plot elements.
    weights : vector or key in ``data``
        If provided, weight the contribution of the corresponding data points
        towards the count in each bin by these factors.
    stat : str
        Aggregate statistic to compute in each bin.

        - `count`: show the number of observations in each bin
        - `frequency`: show the number of observations divided by the bin width
        - `probability` or `proportion`: normalize such that bar heights sum to 1
        - `percent`: normalize such that bar heights sum to 100
        - `density`: normalize such that the total area of the histogram equals 1
    bins : str, number, vector, or a pair of such values
        Generic bin parameter that can be the name of a reference rule,
        the number of bins, or the breaks of the bins.
        Passed to :func:`numpy.histogram_bin_edges`.
    binwidth : number or pair of numbers
        Width of each bin, overrides ``bins`` but can be used with
        ``binrange``.
    binrange : pair of numbers or a pair of pairs
        Lowest and highest value for bin edges; can be used either
        with ``bins`` or ``binwidth``. Defaults to data extremes.
    discrete : bool
        If True, default to ``binwidth=1`` and draw the bars so that they are
        centered on their corresponding data points. This avoids "gaps" that may
        otherwise appear when using discrete (integer) data.
    cumulative : bool
        If True, plot the cumulative counts as bins increase.
    common_bins : bool
        If True, use the same bins when semantic variables produce multiple
        plots. If using a reference rule to determine the bins, it will be comput
ed
        with the full dataset.
    common_norm : bool
        If True and using a normalized statistic, the normalization will apply ov
er
        the full dataset. Otherwise, normalize each histogram independently.
    multiple : {"layer", "dodge", "stack", "fill"}
        Approach to resolving multiple elements when semantic mapping creates sub
sets.
        Only relevant with univariate data.
    element : {"bars", "step", "poly"}
        Visual representation of the histogram statistic.
        Only relevant with univariate data.
    fill : bool
        If True, fill in the space under the histogram.
        Only relevant with univariate data.
    shrink : number
        Scale the width of each bar relative to the binwidth by this factor.
        Only relevant with univariate data.
    kde : bool
        If True, compute a kernel density estimate to smooth the distribution
        and show on the plot as (one or more) line(s).
        Only relevant with univariate data.
```

```
    kde_kws : dict
        Parameters that control the KDE computation, as in :func:`kdeplot`.
    line_kws : dict
        Parameters that control the KDE visualization, passed to
        :meth:`matplotlib.axes.Axes.plot`.
    thresh : number or None
        Cells with a statistic less than or equal to this value will be transpare
nt.
        Only relevant with bivariate data.
    pthresh : number or None
        Like ``thresh``, but a value in [0, 1] such that cells with aggregate cou
nts
        (or other statistics, when used) up to this proportion of the total will
be
        transparent.
    pmax : number or None
        A value in [0, 1] that sets that saturation point for the colormap at a v
alue
        such that cells below constitute this proportion of the total count (or
        other statistic, when used).
    cbar : bool
        If True, add a colorbar to annotate the color mapping in a bivariate plo
t.
        Note: Does not currently support plots with a ``hue`` variable well.
    cbar_ax : :class:`matplotlib.axes.Axes`
        Pre-existing axes for the colorbar.
    cbar_kws : dict
        Additional parameters passed to :meth:`matplotlib.figure.Figure.colorbar
`.
    palette : string, list, dict, or :class:`matplotlib.colors.Colormap`
        Method for choosing the colors to use when mapping the ``hue`` semantic.
        String values are passed to :func:`color_palette`. List or dict values
        imply categorical mapping, while a colormap object implies numeric mappin
g.
    hue_order : vector of strings
        Specify the order of processing and plotting for categorical levels of th
e
        ``hue`` semantic.
    hue_norm : tuple or :class:`matplotlib.colors.Normalize`
        Either a pair of values that set the normalization range in data units
        or an object that will map from data units into a [0, 1] interval. Usage
        implies numeric mapping.
    color : :mod:`matplotlib color <matplotlib.colors>`
        Single color specification for when hue mapping is not used. Otherwise, t
he
        plot will try to hook into the matplotlib property cycle.
    log_scale : bool or number, or pair of bools or numbers
        Set axis scale(s) to log. A single value sets the data axis for any numer
ic
        axes in the plot. A pair of values sets each axis independently.
        Numeric values are interpreted as the desired base (default 10).
        When `None` or `False`, seaborn defers to the existing Axes scale.
    legend : bool
        If False, suppress the legend for semantic variables.
    ax : :class:`matplotlib.axes.Axes`
        Pre-existing axes for the plot. Otherwise, call :func:`matplotlib.pyplot.
gca`
        internally.
    kwargs
        Other keyword arguments are passed to one of the following matplotlib
```

```
        functions:

        - :meth:`matplotlib.axes.Axes.bar` (univariate, element="bars")
        - :meth:`matplotlib.axes.Axes.fill_between` (univariate, other element, f
ill=True)
        - :meth:`matplotlib.axes.Axes.plot` (univariate, other element, fill=Fals
e)
        - :meth:`matplotlib.axes.Axes.pcolormesh` (bivariate)

    Returns
    -------
    :class:`matplotlib.axes.Axes`
        The matplotlib axes containing the plot.

    See Also
    --------
    displot : Figure-level interface to distribution plot functions.
    kdeplot : Plot univariate or bivariate distributions using kernel density est
imation.
    rugplot : Plot a tick at each observation value along the x and/or y axes.
    ecdfplot : Plot empirical cumulative distribution functions.
    jointplot : Draw a bivariate plot with univariate marginal distributions.

    Notes
    -----

    The choice of bins for computing and plotting a histogram can exert
    substantial influence on the insights that one is able to draw from the
    visualization. If the bins are too large, they may erase important features.
    On the other hand, bins that are too small may be dominated by random
    variability, obscuring the shape of the true underlying distribution. The
    default bin size is determined using a reference rule that depends on the
    sample size and variance. This works well in many cases, (i.e., with
    "well-behaved" data) but it fails in others. It is always a good to try
    different bin sizes to be sure that you are not missing something important.
    This function allows you to specify bins in several different ways, such as
    by setting the total number of bins to use, the width of each bin, or the
    specific locations where the bins should break.

    Examples
    --------

    .. include:: ../docstrings/histplot.rst
```

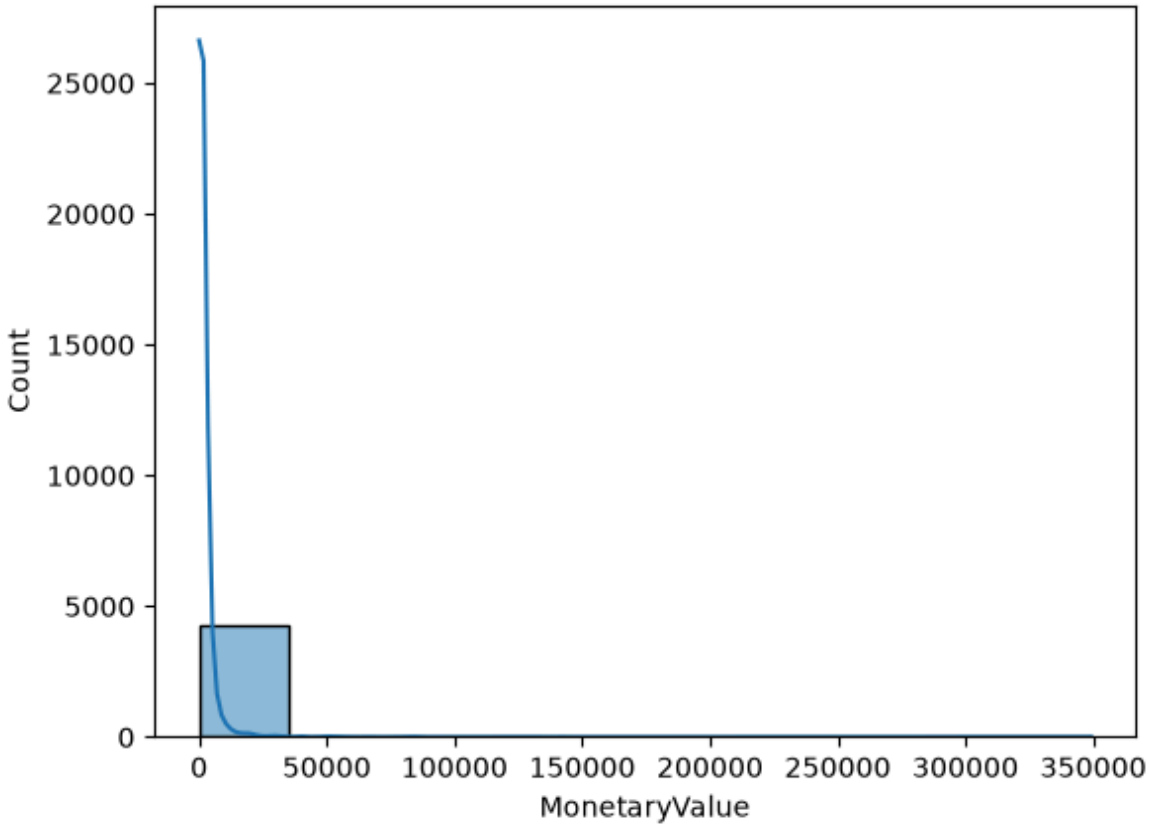
HISTOGRAMS

1. Monetary Value

In [37]:

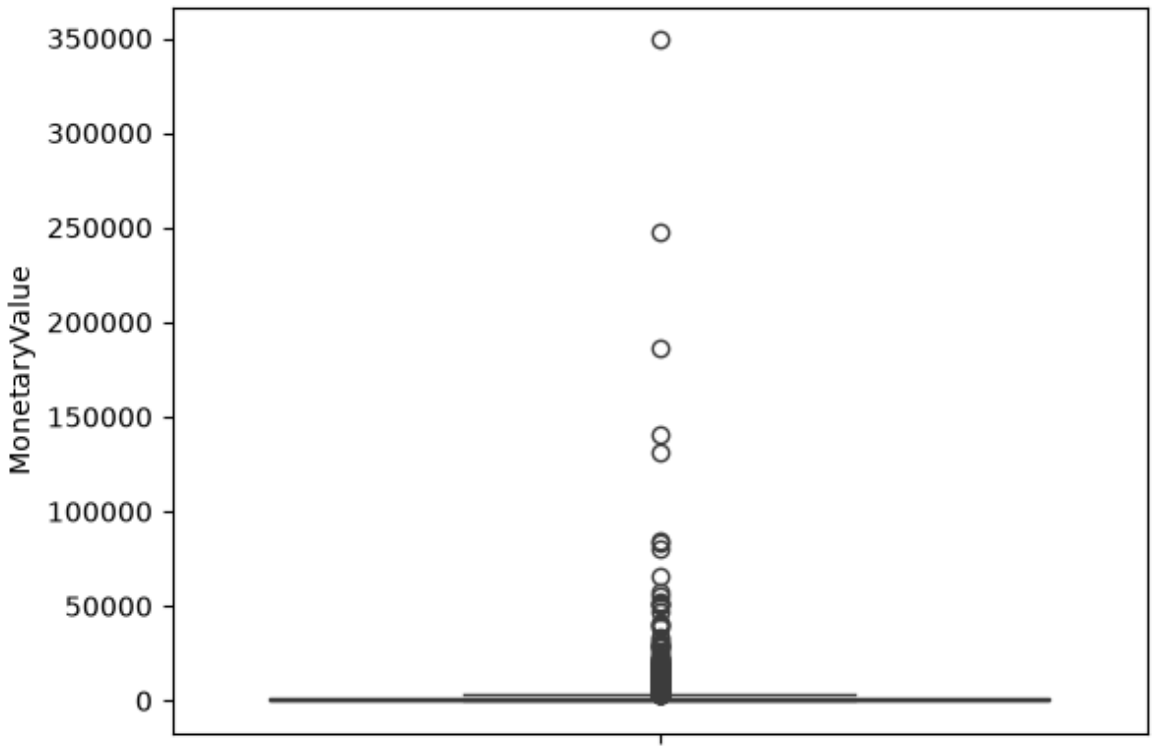
```
sns.histplot(
    data=aggregated_df,
    x="MonetaryValue",
    bins=10,
    kde=True
)

plt.show()
```



In [38]:
```
sns.boxplot(
    data=aggregated_df,
    y="MonetaryValue"
)

plt.show()
```

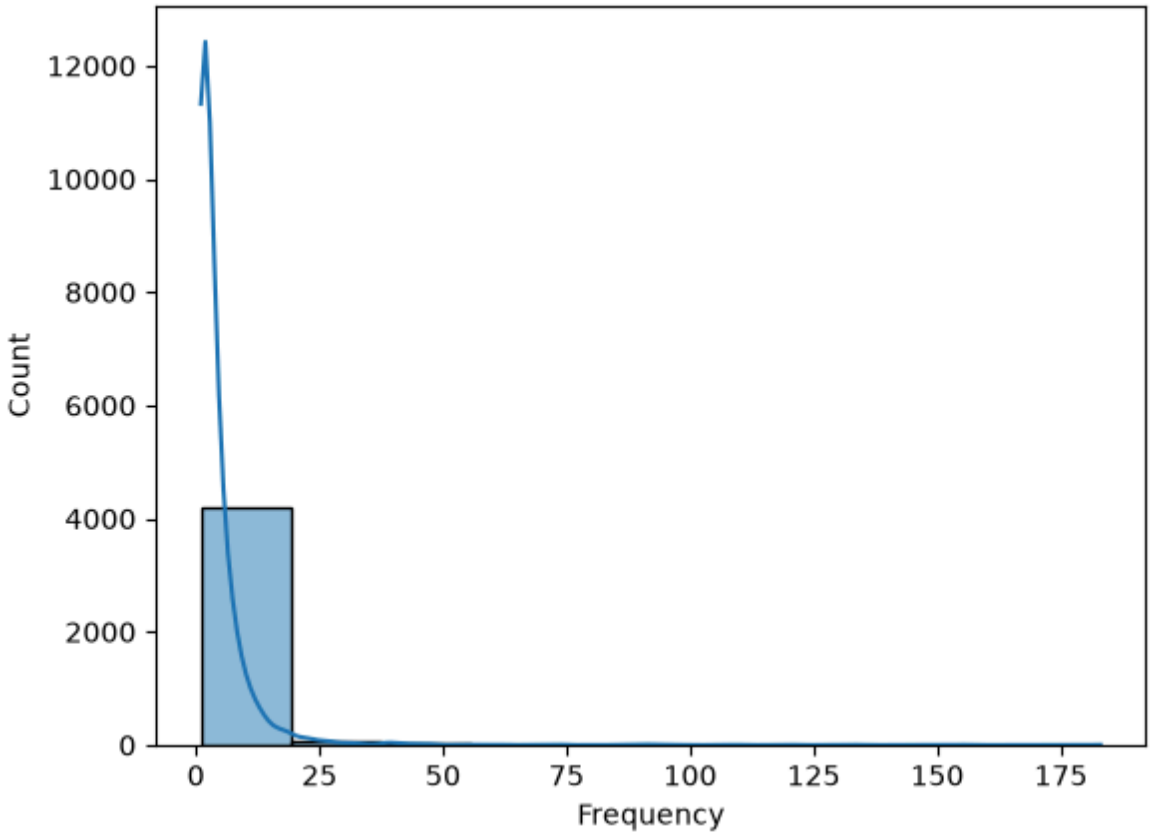


There are so many outliers present in Monetary Value column.

2. Frequency Value

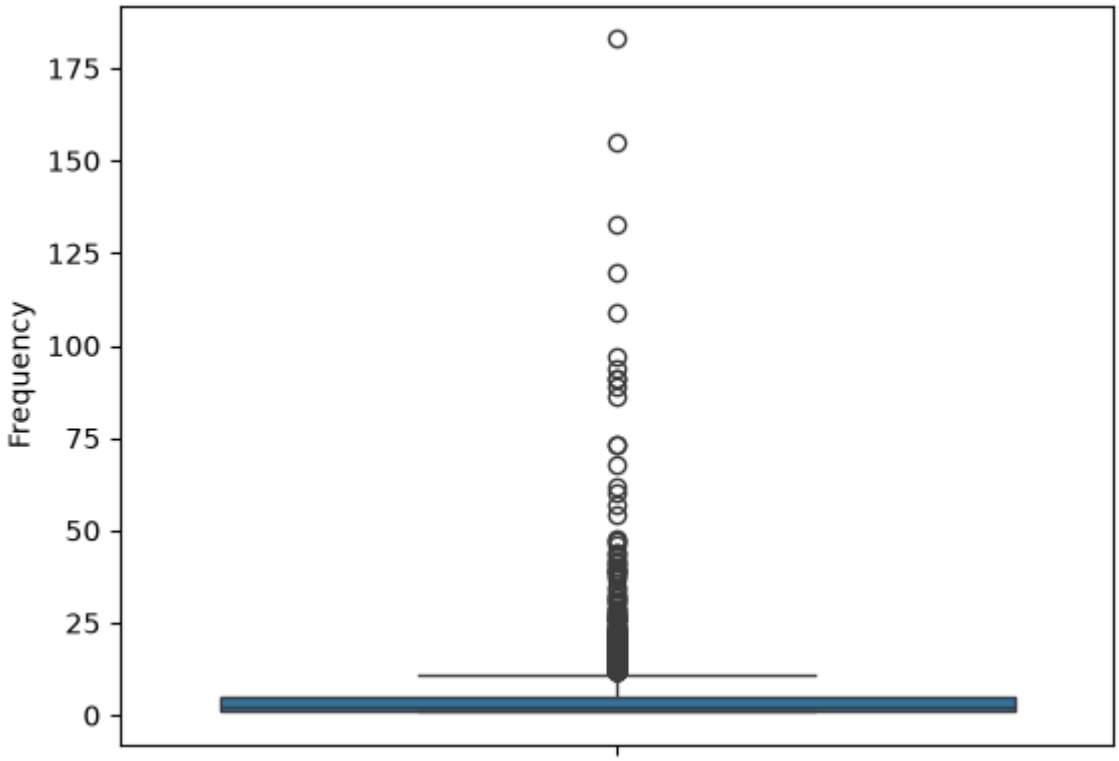
In [39]:
```
sns.histplot(
    data=aggregated_df,
    x="Frequency",
    bins=10,
    kde=True
)

plt.show()
```



In [40]:
```
sns.boxplot(
    data=aggregated_df,
    y="Frequency"
)

plt.show()
```

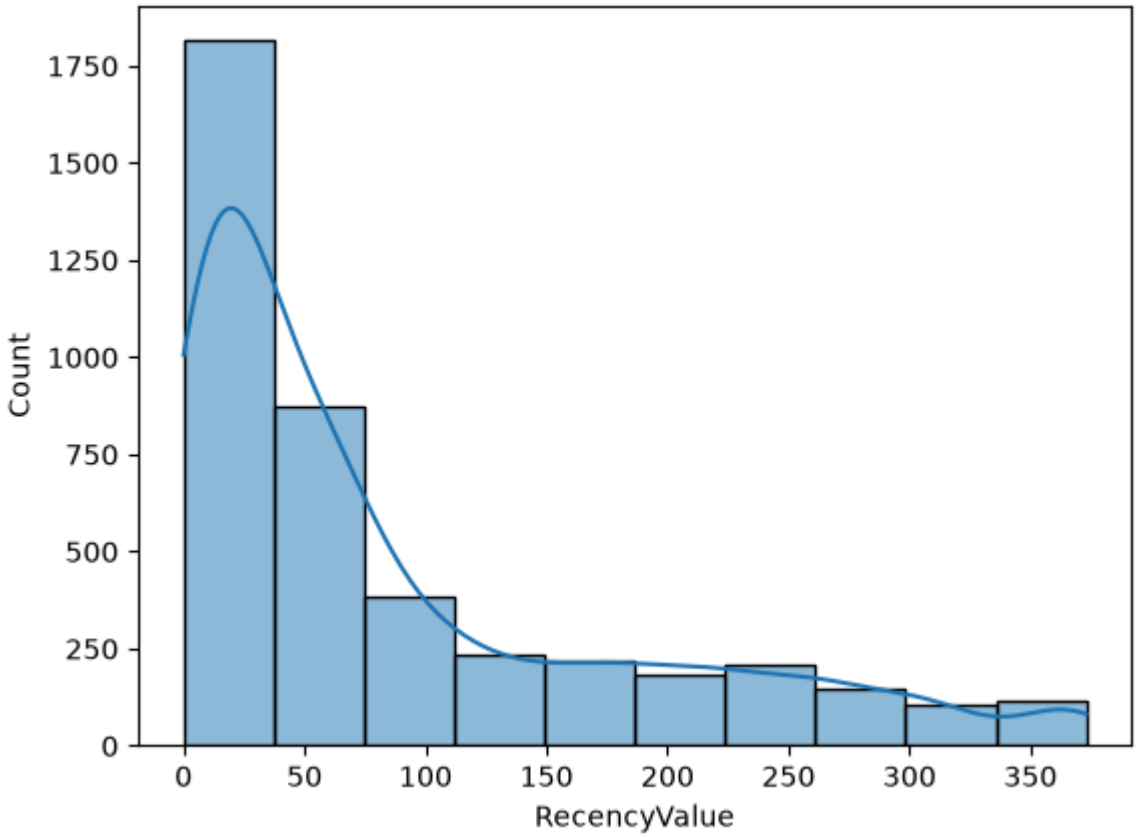


There are several outliers present in this column as well. We will look into this later on.

3. Recency Value

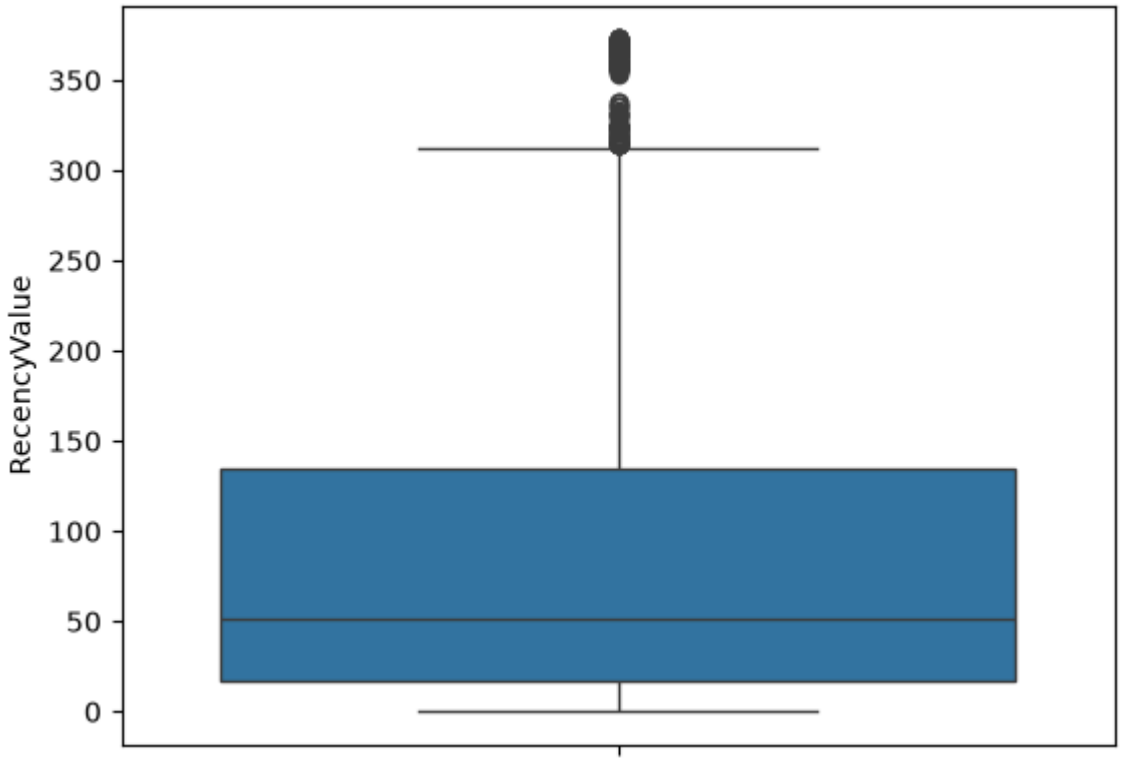
In [41]:
```
sns.histplot(
    data=aggregated_df,
    x="RecencyValue",
    bins=10,
    kde=True
)

plt.show()
```



In [42]:
```
sns.boxplot(
    data=aggregated_df,
    y='RecencyValue'
)

plt.show()
```



There are considerably lesser outliers in this column compared to the other two.

Now as for analysism, we cant simply remove these outliers as these are the most valuable customers in our business problem.

But Outliers are bad for Clustering problem as well. So we will perform the clustering on general customers and perform seperate analysis on outliers.

Lets get hold of the outliers data.

In [43]:
```
# Monetary Outliers

M_Q3 = aggregated_df['MonetaryValue'].quantile(0.75)
M_Q1 = aggregated_df['MonetaryValue'].quantile(0.25)
M_IQR = M_Q3 - M_Q1

M_UL = M_Q3 + (1.5 * M_IQR)
M_LL = M_Q1 - (1.5 * M_IQR)

M_outliers = aggregated_df[(aggregated_df['MonetaryValue'] > M_UL) | (aggregated

M_outliers.head()
```

Out[43]:

	Customer ID	MonetaryValue	Frequency	LastInvoiceDate	RecencyValue
9	12357.0	11229.99	1	2010-11-16 10:05:00	23
25	12380.0	4782.84	4	2010-08-31 14:54:00	100
42	12409.0	12346.62	4	2010-10-15 10:24:00	55
48	12415.0	19468.84	4	2010-11-29 15:07:00	10
61	12431.0	4145.52	11	2010-12-01 10:03:00	8

In [44]:
```
M_outliers.describe()
```

Out[44]:

	Customer ID	MonetaryValue	Frequency	LastInvoiceDate	RecencyValue
count	423.000000	423.000000	423.000000	423	423.000000
mean	15103.040189	12188.101740	17.167849	2010-11-09 12:26:02.978723	30.035461
min	12357.000000	3802.040000	1.000000	2009-12-10 18:03:00	0.000000
25%	13622.000000	4605.940000	8.000000	2010-11-08 13:17:30	3.000000
50%	14961.000000	6191.320000	12.000000	2010-11-26 12:19:00	13.000000
75%	16692.000000	10273.235000	18.000000	2010-12-06 10:34:30	31.000000
max	18260.000000	349164.350000	183.000000	2010-12-09 19:32:00	364.000000
std	1728.661406	25830.851045	19.729755	NaN	51.541004

In [45]:
```
# Frequency Outliers

F_Q3 = aggregated_df['Frequency'].quantile(0.75)
F_Q1 = aggregated_df['Frequency'].quantile(0.25)
```

```
F_IQR = F_Q3 - F_Q1
F_UL = F_Q3 + (1.5 * F_IQR)
F_LL = F_Q3 - (1.5 * F_IQR)

F_outliers = aggregated_df[(aggregated_df['Frequency'] > F_UL) | (aggregated_df[

F_outliers.head()
```

Out[45]:

	Customer ID	MonetaryValue	Frequency	LastInvoiceDate	RecencyValue
65	12437.0	6834.99	20	2010-11-09 14:46:00	30
84	12471.0	17721.45	44	2010-11-30 14:35:00	9
85	12472.0	10426.48	13	2010-12-05 14:19:00	4
92	12482.0	21941.72	27	2010-05-12 16:51:00	211
115	12523.0	2330.38	12	2010-11-30 12:31:00	9

In [46]:
```
F_outliers.describe()
```

Out[46]:

	Customer ID	MonetaryValue	Frequency	LastInvoiceDate	RecencyValue
count	279.000000	279.000000	279.000000	279	279.000000
mean	15352.655914	14409.714932	23.810036	2010-11-23 11:06:20.645161	16.089606
min	12437.000000	1094.390000	12.000000	2010-05-12 16:51:00	0.000000
25%	13800.000000	4331.560500	13.000000	2010-11-20 13:14:30	2.000000
50%	15465.000000	6615.770000	17.000000	2010-12-02 10:46:00	7.000000
75%	16828.500000	11692.405000	23.000000	2010-12-07 11:08:30	19.000000
max	18260.000000	349164.350000	183.000000	2010-12-09 19:32:00	211.000000
std	1748.429987	31381.736785	21.932937	NaN	26.589117

There can be an overlap between the outliers of Monetary value and the frequency value. We will look into it later on.

In [47]:
```
# Non-Outliers data

non_outliers_df = aggregated_df[(~aggregated_df.index.isin(M_outliers.index)) &

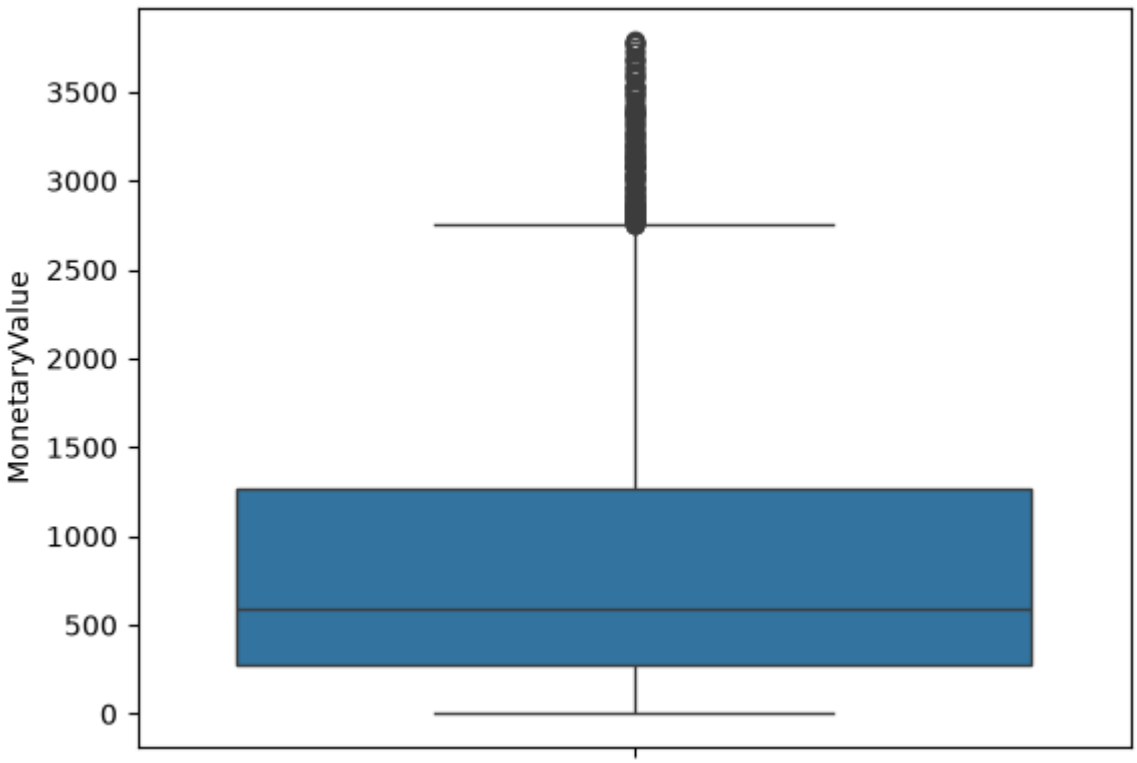
non_outliers_df.describe()
```

Out[47]:

	Customer ID	MonetaryValue	Frequency	LastInvoiceDate	RecencyValue
count	3809.000000	3809.000000	3809.000000	3809	3809.000000
mean	15376.482804	885.502197	2.863481	2010-09-03 11:16:46.516146	97.082174
min	12346.000000	1.550000	1.000000	2009-12-01 10:49:00	0.000000
25%	13912.000000	279.910000	1.000000	2010-07-08 14:48:00	22.000000
50%	15389.000000	588.050000	2.000000	2010-10-12 16:25:00	58.000000
75%	16854.000000	1269.050000	4.000000	2010-11-17 13:14:00	154.000000
max	18287.000000	3788.210000	11.000000	2010-12-09 20:01:00	373.000000
std	1693.199806	817.672283	2.243748	NaN	98.111495

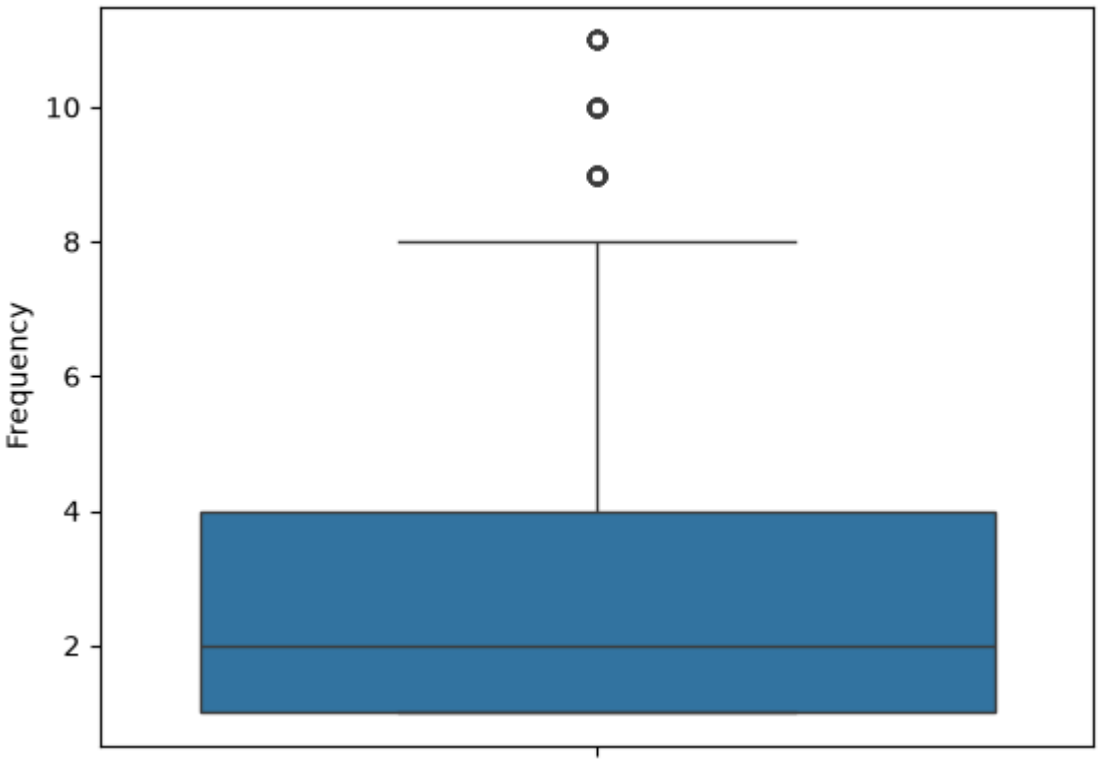
In [48]:
```
sns.boxplot(
    data=non_outliers_df,
    y="MonetaryValue"
)

plt.show()
```



In [49]:
```
sns.boxplot(
    data=non_outliers_df,
    y="Frequency"
)
```

```
plt.show()
```



Now the outliers are seem to be much more bearable.

Now, For Clustering we will be using K-Means Clustering Algorithm. K-Means clustering is a distance based algorithm where scales of different entities matter a lot.

So we will perform a Standard Scaling operation on the data.

In [50]:
```
scaler = StandardScaler()

scaled_data = scaler.fit_transform(non_outliers_df[["MonetaryValue", "Frequency"

scaled_data
```

Out[50]:
```
array([[-0.87594534, -0.38488934,  0.68214853],
       [ 0.5355144 , -0.38488934, -0.96925093],
       [-0.81258645, -0.83063076, -0.24548944],
       ...,
       [-0.62197163, -0.83063076,  2.01753946],
       [ 0.44146683, -0.38488934,  0.14187587],
       [ 1.72488781,  0.50659348, -0.81634357]], shape=(3809, 3))
```

We recieved a Numpy array here. We will convert this to a dataframe

In [51]:
```
scaled_data_df = pd.DataFrame(scaled_data, index=non_outliers_df.index, columns
```

In [52]:
```
scaled_data_df
```

Out[52]:

	MonetaryValue	FrequencyValue	RecencyValue
0	-0.875945	-0.384889	0.682149
1	0.535514	-0.384889	-0.969251
2	-0.812586	-0.830631	-0.245489
3	1.633678	-0.384889	-0.561498
4	-0.715016	-0.830631	-0.887700
...	...	...	...
4280	-0.298120	1.398076	-0.816344
4281	-0.579553	-0.830631	-0.316846
4282	-0.621972	-0.830631	2.017539
4283	0.441467	-0.384889	0.141876
4284	1.724888	0.506593	-0.816344

3809 rows × 3 columns

K-Means Clustering

We will identify the value of K using the infamous "Elbow Method"

In [53]:
```
max_k = 12

inertia = []
silhouette_scores =[]
k_vals = range(2, max_k + 1)

for k in k_vals:

    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        max_iter=1000
    )

    cluster_labels = kmeans.fit_predict(scaled_data_df)

    sil_score = silhouette_score(scaled_data_df, cluster_labels)

    silhouette_scores.append(sil_score)

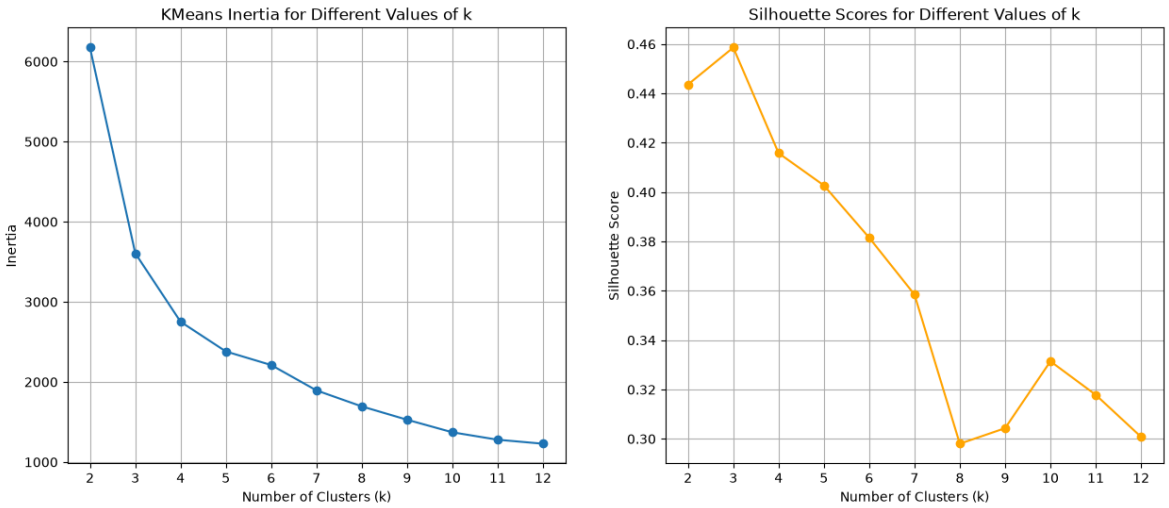
    inertia.append(kmeans.inertia_)
plt.figure(figsize=(15,6))


plt.subplot(1, 2, 1)
plt.plot(k_vals, inertia, marker='o')
plt.title('KMeans Inertia for Different Values of k')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia')
plt.xticks(k_vals)
```

```
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(k_vals, silhouette_scores, marker='o', color='orange')
plt.title('Silhouette Scores for Different Values of k')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Silhouette Score')
plt.xticks(k_vals)
plt.grid(True)


plt.grid(True)
plt.show()
```



4,5 Appears to be best k Value using the elbow method. To determine the best one we will use another metric Which is the "Silhouette Score".

We can see that k=4 has better silhouette score. We will go ahead and use 4.

In [54]:
```
# Final Model

kmeans = KMeans(
    n_clusters=4,
    random_state=42,
    max_iter=1000
)

cluster_labels = kmeans.fit_predict(scaled_data_df)

cluster_labels
```

Out[54]:
```
array([1, 0, 2, ..., 1, 0, 0], shape=(3809,), dtype=int32)
```

Now, we have eaech customer assigned to one particular cluster.

Cluster Analysis

In [55]:
```
non_outliers_df["Cluster"] = cluster_labels

non_outliers_df
```

Out[55]:

	Customer ID	MonetaryValue	Frequency	LastInvoiceDate	RecencyValue	Cluster
0	12346.0	169.36	2	2010-06-28 13:53:00	164	1
1	12347.0	1323.32	2	2010-12-07 14:57:00	2	0
2	12348.0	221.16	1	2010-09-27 14:59:00	73	2
3	12349.0	2221.14	2	2010-10-28 08:23:00	42	0
4	12351.0	300.93	1	2010-11-29 15:23:00	10	2
...	...	...	...	...	...	...
4280	18283.0	641.77	6	2010-11-22 15:30:00	17	0
4281	18284.0	411.68	1	2010-10-04 11:33:00	66	2
4282	18285.0	377.00	1	2010-02-17 10:24:00	295	1
4283	18286.0	1246.43	2	2010-08-20 11:57:00	111	0
4284	18287.0	2295.71	4	2010-11-22 11:51:00	17	0

3809 rows × 6 columns

In [56]:
```
non_outliers_df['Cluster'].unique()
```

Out[56]:
```
array([1, 0, 2, 3], dtype=int32)
```

In [57]:
```
cluster_colors = {
    0: '#1f77b4',  # Blue
    1: '#ff7f0e',  # Orange
    2: '#2ca02c',  # Green
    3: '#d62728'   # Red
}

colors = non_outliers_df['Cluster'].map(cluster_colors)

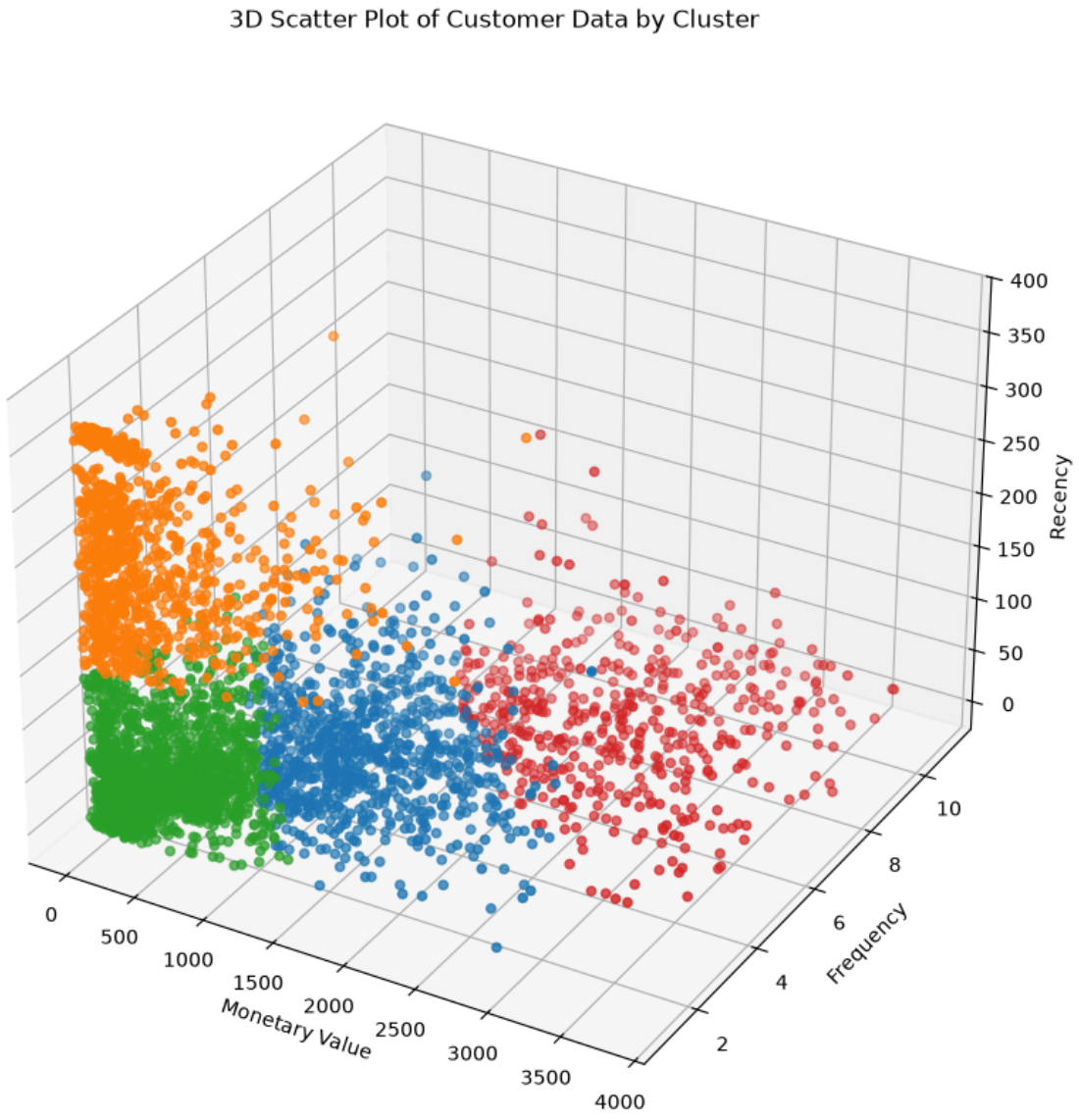
fig = plt.figure(figsize=(10, 10))
ax = fig.add_subplot(projection='3d')

scatter = ax.scatter(
    non_outliers_df['MonetaryValue'],
    non_outliers_df['Frequency'],
    non_outliers_df['RecencyValue'],
    c=colors,  # Use mapped solid colors
    marker='o'
)
```

```
ax.set_xlabel('Monetary Value')
ax.set_ylabel('Frequency')
ax.set_zlabel('Recency')

ax.set_title('3D Scatter Plot of Customer Data by Cluster')

plt.show()
```



In [58]:
```
plt.figure(figsize=(12, 18))

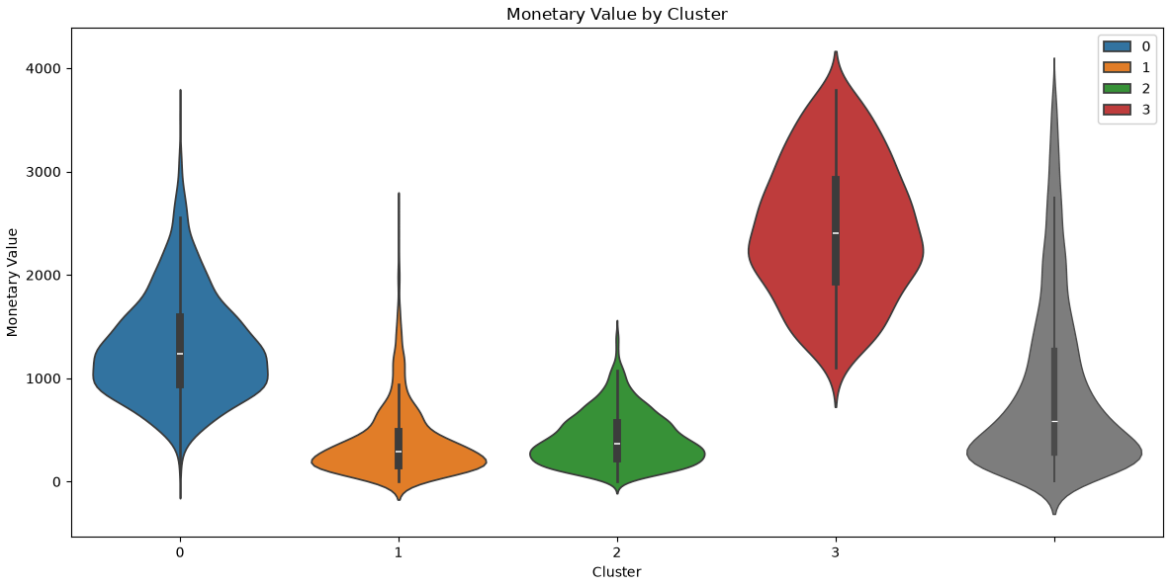
plt.subplot(3, 1, 1)
sns.violinplot(x=non_outliers_df['Cluster'], y=non_outliers_df['MonetaryValue'],
sns.violinplot(y=non_outliers_df['MonetaryValue'], color='gray', linewidth=1.0)
plt.title('Monetary Value by Cluster')
plt.ylabel('Monetary Value')

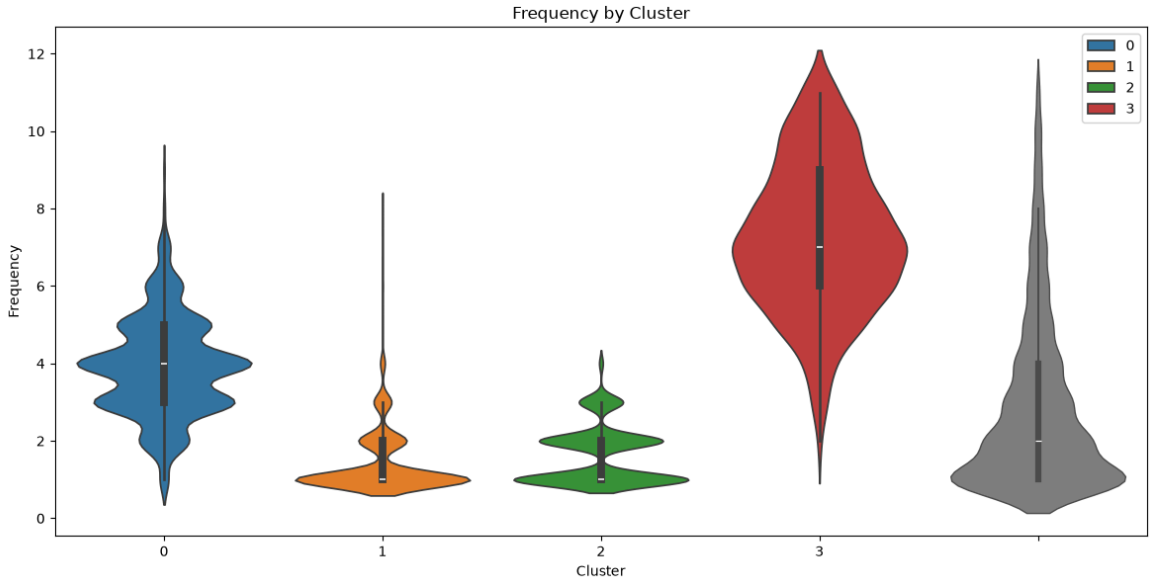
plt.subplot(3, 1, 2)
sns.violinplot(x=non_outliers_df['Cluster'], y=non_outliers_df['Frequency'], pal
sns.violinplot(y=non_outliers_df['Frequency'], color='gray', linewidth=1.0)
plt.title('Frequency by Cluster')
plt.ylabel('Frequency')

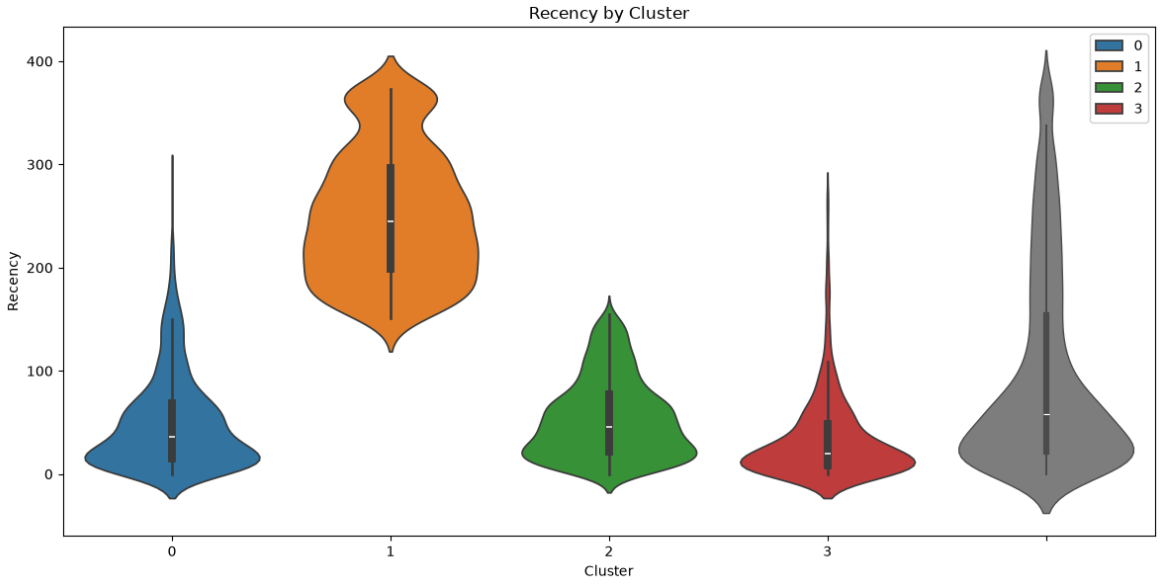

plt.subplot(3, 1, 3)
sns.violinplot(x=non_outliers_df['Cluster'], y=non_outliers_df['RecencyValue'],
```

```
sns.violinplot(y=non_outliers_df['RecencyValue'], color='gray', linewidth=1.0)
plt.title('Recency by Cluster')
plt.ylabel('Recency')

plt.tight_layout()
plt.show()
```







With this information we need to get names of these clusters.

1. Cluster 0 (Blue): "Retain"

- Rationale: This cluster represents high-value customers who purchase regularly, though not always very recently. The focus should be on retention efforts to maintain their loyalty and spending levels.
- Action: Implement loyalty programs, personalized offers, and regular engagement to ensure they remain active.

2. Cluster 1 (Orange): "Re-Engage"

- Rationale: This group includes lower-value, infrequent buyers who haven't purchased recently. The focus should be on re-engagement to bring them back into active purchasing behavior.
- Action: Use targeted marketing campaigns, special discounts, or reminders to encourage them to return and purchase again.

3. Cluster 2 (Green): "Nurture"

- Rationale: This cluster represents the least active and lowest-value customers, but they have made recent purchases. These customers may be new or need nurturing to increase their engagement and spending.
- Action: Focus on building relationships, providing excellent customer service, and offering incentives to encourage more frequent purchases.

4. Cluster 3 (Red): "Reward"

- Rationale: This cluster includes high-value, very frequent buyers, many of whom are still actively purchasing. They are your most loyal customers, and rewarding their loyalty is key to maintaining their engagement.
- Action: Implement a robust loyalty program, provide exclusive offers, and recognize their loyalty to keep them engaged and satisfied.

Summary of Cluster Names:

- Cluster 0 (Blue): "Retain"
- Cluster 1 (Orange): "Re-Engage"
- Cluster 2 (Green): "Nurture"
- Cluster 3 (Red): "Reward"

OUTLIERS ANALYSIS

In [59]:
```
overlap_indices = M_outliers.index.intersection(F_outliers.index)
```

In [60]:
```
overlap_indices
```

Out[60]:
```
Index([  65,   84,   85,   92,  126,  145,  164,  179,  202,  203,
       ...
       4107, 4125, 4143, 4156, 4200, 4235, 4236, 4237, 4241, 4262],
      dtype='int64', length=226)
```

In [62]:
```
monetary_only_outliers = M_outliers.drop(overlap_indices)
frequency_only_outliers = F_outliers.drop(overlap_indices)

monetary_and_frequency_outliers = M_outliers.loc[overlap_indices]
```

In [63]:
```
# Now we will manually assign cluster labels for these.

monetary_only_outliers['Cluster'] = -1
frequency_only_outliers["Cluster"] = -2
monetary_and_frequency_outliers["Cluster"] = -3
```

In [64]:
```
outlier_clusters_df = pd.concat([monetary_only_outliers, frequency_only_outliers
```

In [65]:
```
outlier_clusters_df
```

Out[65]:

	Customer ID	MonetaryValue	Frequency	LastInvoiceDate	RecencyValue	Cluster
9	12357.0	11229.99	1	2010-11-16 10:05:00	23	-1
25	12380.0	4782.84	4	2010-08-31 14:54:00	100	-1
42	12409.0	12346.62	4	2010-10-15 10:24:00	55	-1
48	12415.0	19468.84	4	2010-11-29 15:07:00	10	-1
61	12431.0	4145.52	11	2010-12-01 10:03:00	8	-1
...	...	...	...	...	...	...
4235	18223.0	7516.31	12	2010-11-17 12:20:00	22	-3
4236	18225.0	7545.14	15	2010-12-09 15:46:00	0	-3
4237	18226.0	6650.83	15	2010-11-26 15:51:00	13	-3
4241	18231.0	4791.80	23	2010-10-29 14:17:00	41	-3
4262	18260.0	7318.91	17	2010-11-30 12:25:00	9	-3

476 rows × 6 columns

In [68]:
```
cluster_colors = {-1: '#9467bd',
                  -2: '#8c564b',
                  -3: '#e377c2'}
```

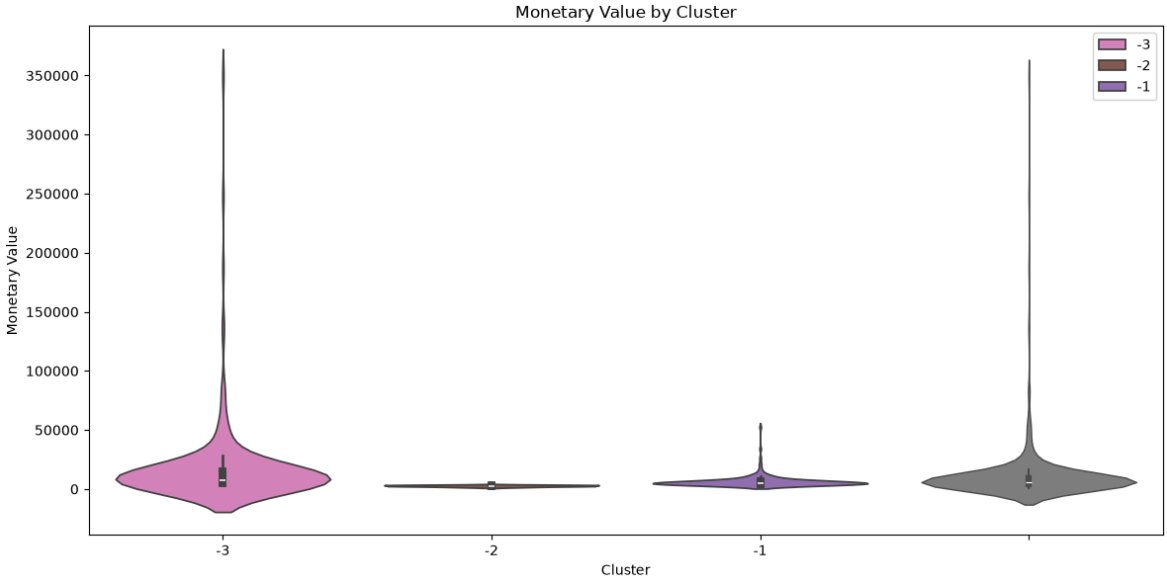
```
plt.figure(figsize=(12, 18))

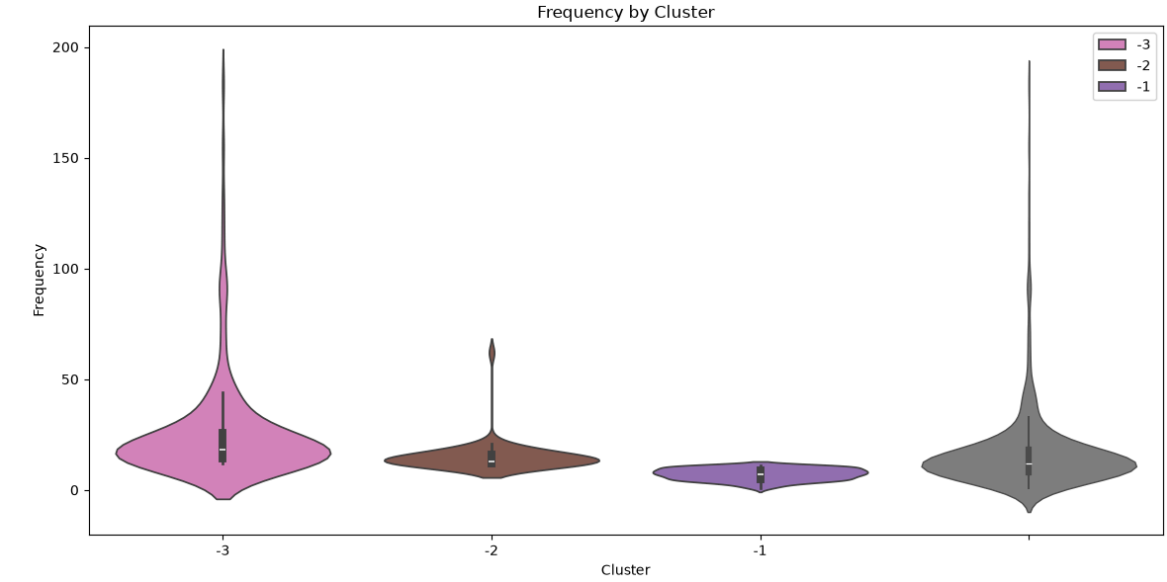
plt.subplot(3, 1, 1)
sns.violinplot(x=outlier_clusters_df['Cluster'], y=outlier_clusters_df['Monetary
sns.violinplot(y=outlier_clusters_df['MonetaryValue'], color='gray', linewidth=1
plt.title('Monetary Value by Cluster')
plt.ylabel('Monetary Value')

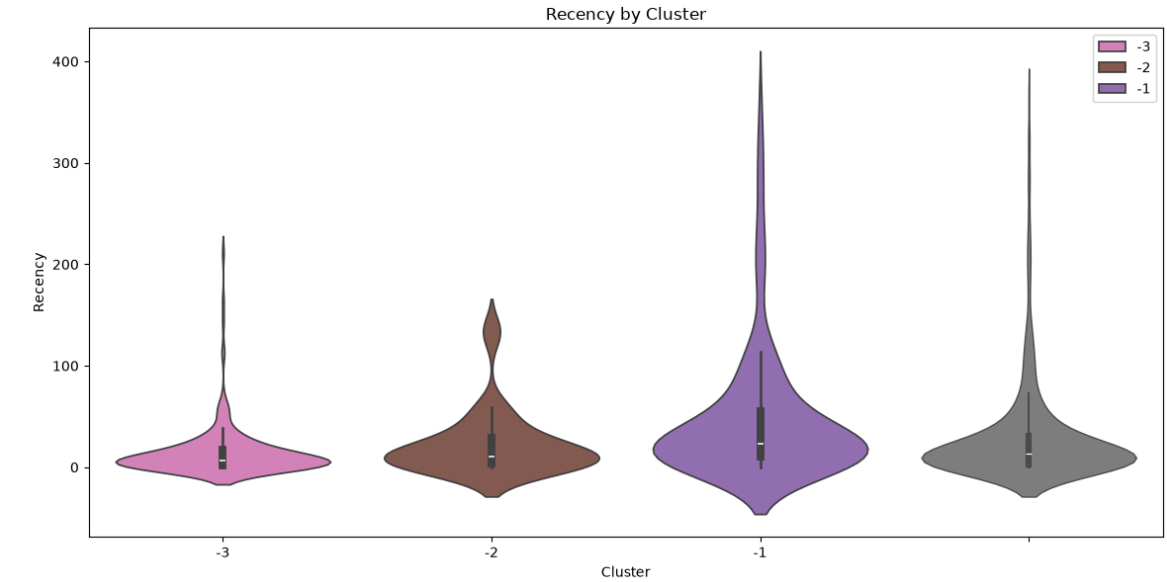
plt.subplot(3, 1, 2)
sns.violinplot(x=outlier_clusters_df['Cluster'], y=outlier_clusters_df['Frequenc
sns.violinplot(y=outlier_clusters_df['Frequency'], color='gray', linewidth=1.0)
plt.title('Frequency by Cluster')
plt.ylabel('Frequency')

plt.subplot(3, 1, 3)
sns.violinplot(x=outlier_clusters_df['Cluster'], y=outlier_clusters_df['RecencyV
sns.violinplot(y=outlier_clusters_df['RecencyValue'], color='gray', linewidth=1.
plt.title('Recency by Cluster')
plt.ylabel('Recency')

plt.tight_layout()
plt.show()
```







Cluster -1 (Monetary Outliers) PAMPER: Characteristics: High spenders but not necessarily frequent buyers. Their purchases are large but infrequent. Potential Strategy: Focus on maintaining their loyalty with personalized offers or luxury services that cater to their high spending capacity.

Cluster -2 (Frequency Outliers) UPSELL: Characteristics: Frequent buyers who spend less per purchase. These customers are consistently engaged but might benefit from upselling opportunities. Potential Strategy: Implement loyalty programs or bundle deals to encourage higher spending per visit, given their frequent engagement.

Cluster -3 (Monetary & Frequency Outliers) DELIGHT: Characteristics: The most valuable outliers, with extreme spending and frequent purchases. They are likely your top-tier customers who require special attention. Potential Strategy: Develop VIP programs or exclusive offers to maintain their loyalty and encourage continued engagement.

In [69]:
```
# Final Cluster Labels

cluster_labels = {
    0: "RETAIN",
    1: "RE-ENGAGE",
    2: "NURTURE",
    3: "REWARD",
    -1: "PAMPER",
    -2: "UPSELL",
    -3: "DELIGHT"
}
```

In [70]:
```
full_clustering_df = pd.concat([non_outliers_df, outlier_clusters_df])

full_clustering_df
```

Out[70]:

	Customer ID	MonetaryValue	Frequency	LastInvoiceDate	RecencyValue	Cluster
0	12346.0	169.36	2	2010-06-28 13:53:00	164	1
1	12347.0	1323.32	2	2010-12-07 14:57:00	2	0
2	12348.0	221.16	1	2010-09-27 14:59:00	73	2
3	12349.0	2221.14	2	2010-10-28 08:23:00	42	0
4	12351.0	300.93	1	2010-11-29 15:23:00	10	2
...	...	...	...	...	...	...
4235	18223.0	7516.31	12	2010-11-17 12:20:00	22	-3
4236	18225.0	7545.14	15	2010-12-09 15:46:00	0	-3
4237	18226.0	6650.83	15	2010-11-26 15:51:00	13	-3
4241	18231.0	4791.80	23	2010-10-29 14:17:00	41	-3
4262	18260.0	7318.91	17	2010-11-30 12:25:00	9	-3

4285 rows × 6 columns

In [71]:
```
full_clustering_df['ClusterLabel'] = full_clustering_df["Cluster"].map(cluster_l

full_clustering_df
```

Out[71]:

	Customer ID	MonetaryValue	Frequency	LastInvoiceDate	RecencyValue	Cluster	Cl
0	12346.0	169.36	2	2010-06-28 13:53:00	164	1	F
1	12347.0	1323.32	2	2010-12-07 14:57:00	2	0	
2	12348.0	221.16	1	2010-09-27 14:59:00	73	2	
3	12349.0	2221.14	2	2010-10-28 08:23:00	42	0	
4	12351.0	300.93	1	2010-11-29 15:23:00	10	2	
...	...	...	...	...	...	...	
4235	18223.0	7516.31	12	2010-11-17 12:20:00	22	-3	
4236	18225.0	7545.14	15	2010-12-09 15:46:00	0	-3	
4237	18226.0	6650.83	15	2010-11-26 15:51:00	13	-3	
4241	18231.0	4791.80	23	2010-10-29 14:17:00	41	-3	
4262	18260.0	7318.91	17	2010-11-30 12:25:00	9	-3	

4285 rows × 7 columns

Visualization

In [73]:

```
cluster_counts = full_clustering_df['ClusterLabel'].value_counts()
full_clustering_df["MonetaryValue per 100 pounds"] = full_clustering_df["Monetar
feature_means = full_clustering_df.groupby('ClusterLabel')[['RecencyValue', 'Fre

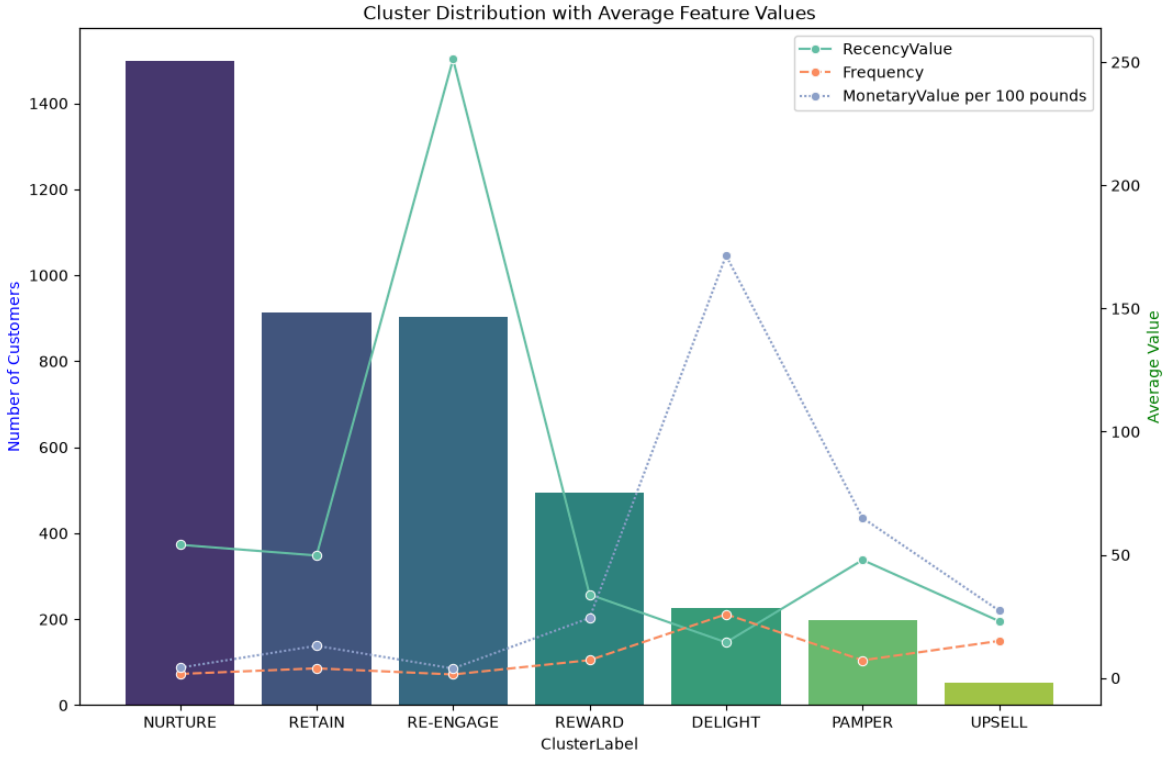
fig, ax1 = plt.subplots(figsize=(12, 8))

sns.barplot(x=cluster_counts.index, y=cluster_counts.values, ax=ax1, palette='vi
ax1.set_ylabel('Number of Customers', color='b')
ax1.set_title('Cluster Distribution with Average Feature Values')

ax2 = ax1.twinx()

sns.lineplot(data=feature_means, ax=ax2, palette='Set2', marker='o')
ax2.set_ylabel('Average Value', color='g')

plt.show()
```


Cluster Distribution with Average Feature Values
RecencyValue
Frequency
MonetaryValue per 100 pounds
Number of Customers
Average Value
0
200
400
600
800
1000
1200
1400
0
50
100
150
200
250
NURTURE
RETAIN
RE-ENGAGE
REWARD
DELIGHT
PAMPER
UPSELL
ClusterLabel

In [ ]: